Aula 11: Bits: XOR, AND e deslocamento

Lógica de Programação – Superior · Bloco 1

José Dihego · IFBA

O que você precisa saber hoje

Objetivo: Manipular os bits de um inteiro com `^`, `&` e `>>`.

- `a ^ b` (XOR): bit 1 onde `a` e `b` diferem
- `x & 1` é o último bit; `x >> k` desloca `k` casas para a direita
- Bit `k` de `x`: `(x >> k) & 1`
- Inverter o bit `k`: `x ^ (1 << k)`
- Equivalente sem bits: `% 2` e `/ 2`

O padrão em C

```c
int x = a ^ b;                  // bits diferentes
int dif = 0;
for (int k = 0; k < 4; k++)
    dif += (x >> k) & 1;        // conta os 1
```

Erros que custam pontos

Errado	Certo	Por quê
`if (x & 1 == 1)`	`if ((x & 1) == 1)`	== tem prioridade maior que &
`a ^ 2 /* "a ao quadrado" */`	`a * a`	^ é XOR, não potência

Teste de mesa

```
int a = 9, b = 3, x = a ^ b, dif = 0;
for (int k = 0; k < 4; k++) {
    dif += (x >> k) & 1;
}
```

k	(x » k) & 1	dif
0	0	0
1	1	1
2	0	1
3	1	2

Questão resolvida em sala

Avaliação 1 – 2026.1, Questão II

`avaliacao_1_ano_2026_1.pdf`

Números n-binariamente distintos

Estratégia:

1. XOR marca os bits diferentes entre a e b
2. Contar os bits 1 do XOR (4 bits)
3. Comparar a contagem com n

Enunciado completo (com figuras e exemplos) no PDF da prova.

Solução

```c
#include <stdio.h>                  // printf e scanf

int main(void) {                    // início do programa
    int n, a, b;                    // limiar n e os dois números (menores que 16)
    printf("n, a e b: ");           // pede os dados
    scanf("%d %d %d", &n, &a, &b);  // lê os três inteiros
    int x = a ^ b;                  // OU exclusivo: bit 1 onde a e b diferem
    int diferentes = (x & 1) + ((x >> 1) & 1) + ((x >> 2) & 1) + ((x >> 3) & 1);  // conta os 4 bits
    if (diferentes >= n) printf("%d e %d são %d-binariamente distintos\n", a, b, n); // bits diferentes suficientes
    else printf("%d e %d não são %d-binariamente distintos\n", a, b, n);  // menos bits diferentes que n
    return 0;                       // fim sem erro
}
```

Sua vez: questão proposta

Avaliação 1 – 2023.2, Questão II

`avaliacao_1_ano_2023_2.pdf`

Mutação por swap (inverter um bit)

Dicas:

- Inverter um bit: XOR com `1 << k`
- Cuidado com a numeração das posições (da direita, a partir de 0 ou 1?)
- Imprima em binário para conferir

Para conferir: entrada `28 3` → a saída deve conter `20`

A solução comentada está na página do curso: https://josedihego.net/logica/#aula-11